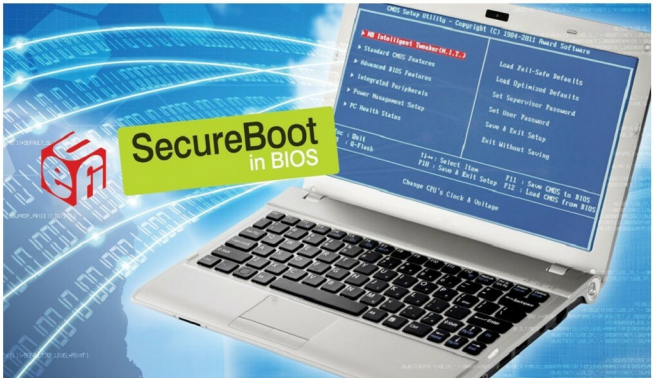


How to Program an **Application for UEFI**

Let us shed a tear for the legacy BIOS. Slowly but surely, UEFI is replacing the legacy BIOS in new computer systems since it has many advantages over the latter. Read on for the reasons why UEFI is preferable to DOS, and also to learn how an application can be programmed in UEFI.



It is a known fact that the transition from the legacy BIOS to UEFI (Unified Extensible Firmware Interface) was considered a radical new approach in the development of personal firmware platforms. This is also a very rare exception, when instead of using assembly language (quite common for these kinds of low-level projects) developers relied upon high-level languages.

Guided by research interest, we decided to go against this trend and explore UEFI at the machine code level. But before we go any further, let's go deeper into pure theory.

UEFI instead of DOS

The UEFI interface sub-system is often compared to DOS. This is due to the fact that UEFI provides the ability to run small applications without booting into a 'big' operating system, and *UEFI Shell*'s functionality resembles the one from DOS. It's time to think about the problems associated with the diagnostics and maintenance of PC, such as a memory test or a BIOS-reflashing. Let's take a look at the advantages of UEFI from the point of view of a systems programmer with a DOS-mentality.

The first advantage: System context

DOS programs usually run in 16-bit real processor mode with only 1 MB address space available. In order to address the rest of the whole memory, the program should create descriptors of its own pages and segments, and switch the CPU into protected mode. At this point, UEFI applications are different, because firmware (i.e., UEFI) itself is responsible for switching CPU into protected mode, as well as for giving context to applications virtual pages that would allow any system object in RAM to be addressed. The application loads into an environment where everything is set up and ready for execution. Please note, that for IA32 EFI architecture, this environment is called *Protected Mode 32*, whilst for x64 UEFI, it is called *Protected Mode 64*.

The second advantage: Support for file operations

The legacy BIOS works with disk devices at the physical sector level, without deciphering the file system contents beneath. That's why OS loading starts by reading the Master Boot Record (MBR), a sector with physical coordinates Cylinder 0, Head 0 and Sector 1. Then, boot control is passed to a program